

Enhanced Reliability Architecture for VLSI Systems Using the IRMC Algorithm for Error Detection and Correction



G. Shanthi, Sk. Shoukath Vali, Uppala Bhargava Sai,
K. L. V. Ramana Kumari, S. Naga Leela, A. Ramesh Kumar,
and K. Srinivas Rao

Abstract The problem of ensuring the integrity of data transmitted on the Internet in the context of modern mobile communication in an environment of increasing complexity transmissions is acute. To ensure protection against degradation due to channel noise and environmental interference, reliable error detection and correction must be used, especially at the data link layer. Communication networks are usually realized through two independent logical channels: a signaling and control channel and a data traffic channel. In the traffic channel, bits of information are then supplemented by an encoded correction code to create a powerful composite data stream that increases fault resilience during transmission. Current SRAM-based memory systems are fault-tolerant to a limited degree, but they are still vulnerable to Single Cell Upsets (SCUs) and Multiple Cell Upsets (MCUs); these are frequent events caused by exposure to high-energy particles, e.g., cosmic radiation. In order to overcome such vulnerabilities, this paper proposes an Improved Redundant Matrix Code (IRMC) method, which is particularly aimed at countering the effects of memory

G. Shanthi · Sk. Shoukath Vali (✉) · U. B. Sai · K. L. V. Ramana Kumari · S. Naga Leela ·
A. Ramesh Kumar
Department of ECE, VNR Vignana Jyothi Institute of Engineering and Technology, Hyderabad,
India
e-mail: shoukathvalishaik@gmail.com

G. Shanthi
e-mail: shanthi_g@vnrvjiet.in

K. L. V. Ramana Kumari
e-mail: ramanakumari_jlv@vnrvjiet.in

S. Naga Leela
e-mail: nagaleela_s@vnrvjiet.in

A. Ramesh Kumar
e-mail: rameshkumar_a@vnrvjiet.in

K. S. Rao
Department of ECE, MEMS Research Center, KLEF, Guntur, India

corruption by powerful detection and correction of various bit faults. Compared to traditional rescue codes, IRMC is highly superior in that it achieves a better detection rate, better correction ability, and reduced overheads in terms of space, power, and delay. Xilinx ISE 14.7 is used to realize the proposed solution, where the behavioral and structural modeling was done in Verilog HDL. Simulation and synthesis confirm IRMC improvement with regard to both error correction strength and hardware efficiency, and signal that it is a promising low-power, high-reliability coding scheme for next-generation memory protection.

Keywords Multiple error detection · Error detection correction code (EDCC) · Improved redundant · Matrix code (IRMC) · Multiple cell upsets (MCU) · Multiple error correction memory

1 Introduction

In this digital age, communication and data transportation systems have become the spine of consumer-oriented services where the users expect uninterrupted, high speed and quality access to data. The parallel data transmission is one of the major enabling strategies whereby users can aggregate and receive data on several channels simultaneously. This is accomplished by sending a set of data streams using multiple dedicated communication lines, and the number of active transmission paths is directly related to the number of bits that are sent in each clock cycle [1].

In practice, the use of parallel transmission requires N discrete channels, and this in turn may lead to higher hardware complexities and costs. Sequential or serial transmission, on the other hand, though having a single channel, is a cheaper and less resource-hungry technology and still does not compromise data fidelity [2].

Regardless of the transmission scheme, data integrity remains a critical concern. As the need to have high-density and high-speed storage grows, memory systems need to make sure that information stored is not affected by errors. Inaccuracies that occur in either transmission or storage of data jeopardize performance and dependability of the system. Among the most prominent causes of this type of fault, called soft errors, are external interferences such as alpha particles or cosmic radiation as opposed to internal design faults [3, 4]. Such radiation-induced phenomena can cause Single Event Upsets (SEUs), such as Single Cell Upsets (SCUs) and Multiple Cell Upsets (MCUs), so that error-control mechanisms are considered essential to terrestrial and space-grade electronic systems.

To help counteract the effects of such soft errors, in modern systems, Error Detection and Correction (EDAC) codes are used. Non-random coding schemes distort the bit structure of transmitted data in a manner that limits the vulnerability to errors. Systematic codes, however, do not alter the original data format but instead add a calculated group of parity or check bits [5]. At the receiver end, the system will perform a parity check so as to identify the difference and maybe correct it. It is the process of the utmost significance that guards the data integrity on the various levels

of the communication protocols and storage media such as the RAM, the hard disks, the CDs, and the DVDs [6–8].

Among the latest advancements in this regard pertain to the Improved Redundant Matrix Code (IRMC)—a coding scheme, which has been devised while having in mind the single objective of effectively detecting as well as correcting memory upsets. The IRMC algorithm focuses on ensuring that the memory components become fault-tolerant to defects that are created in the memory component due to electromagnetic interference or particle hits by enhancing the localization precision of faults, as well as fault correction. IRMC is aimed at Very-Large-Scale Integration (VLSI) systems and is, further, performance-efficient than state-of-the-art in terms of detection accuracy, correction strength, and resource consumption, with power, area, and timing constraints in mind [9, 10].

Despite the availability of several error detection and correction techniques, such as Decimal Matrix Code (DMC), Bit Manipulation Code (BMC), and Reed-Muller Code, these methods often suffer from limitations in terms of scalability, power efficiency, and correction accuracy when applied to high-density VLSI systems. DMC and BMC, for instance, introduce significant overhead in terms of delay and power consumption, making them less suitable for low-power embedded applications. Reed-Muller codes, while robust, are computationally intensive and not easily adaptable to real-time systems. Moreover, existing schemes lack the flexibility to efficiently handle multiple cell upsets (MCUs) in dynamic environments such as space-grade or mobile communication systems.

To address these challenges, this work introduces the Improved Redundant Matrix Code (IRMC) algorithm, which offers enhanced fault localization, reduced resource overhead, and superior correction capability.

By leveraging a dual-dimensional parity structure and optimized syndrome computation, IRMC provides a scalable and energy-efficient solution tailored for next-generation VLSI architectures. This research fills a critical gap by proposing a method that balances reliability, performance, and hardware efficiency—a combination not adequately achieved by current state-of-the-art techniques.

This Chapter presents the Verilog HDL implementation and simulation of the IRMC algorithm using the Xilinx ISE 14.7 toolchain. It must be established that IRMC can offer low-latency and energy energy-efficient, and highly reliable error correction to contemporary memory architectures and embedded systems. The bigger picture is to help design effective communication systems, which would perform well even under poor operating and environmental conditions [11–13].

2 Improved Redundant Matrix Code Algorithm

The Improved Redundant Matrix Code (IRMC) algorithm is applied to enhance the integrity of information by systematically introducing a matrix of redundant bits to the original input information, and thus an encoded codeword is produced, which is capable of detecting and correcting faults. Through the smart design of such a

redundant matrix, the errors are contained and are correctable. The IRMC technique has particular advantages in Very-Large-Scale Integration (VLSI) applications where transistor densities are great and thus vulnerability to soft errors is increased. With IRMC, designers stand a chance of enormously augmenting the dependability of circuits, slashing the probability of data corruption, as well as augmenting the resilience of the entire system [14].

As well as finding application in VLSI applications, IRMC has proven itself to be useful to a wide degree in such fields as telecommunication systems, data storage facilities, and satellite communications, where a high degree of reliability over a wide range of environmental conditions is required [15, 16]. IRMC, with its rate of success and low overhead, is becoming a popular, reliable error correction methodology, and with the ever-increasing spread of technology, its significance is only going to rise. This work implements the IRMC algorithm using Verilog HDL and the Xilinx ISE 14.7 suite to identify and correct errors in 32-bit and 64-bit input streams. Encoding phase. The original 32-bit or 64-bit input is processed to generate Horizontal Redundant Bits using AXL-based computations, and Vertical Redundant Bits are generated using a logical matrix formulation of the input data [17].

The decoding level accepts erroneous data with the same bit-width as the input, and by re-running the redundant bit generation algorithm, it produces a new pair of horizontal and vertical parity bits. These values are then used to come up with a syndrome, which denotes the existence of any errors and their position. When a successful detection is made, the incorrect bit is fixed, and it is replaced with its original value.

3 Encoder of the IRMC Algorithm

The Enhanced Redundancy Matrix Coding algorithm can detect and correct multiple errors in a message. It works by creating a parity check matrix that is used to detect errors in received messages. The same matrix is created by adding additional rows and columns to the original message matrix [18]. These additional rows and columns contain parity bits used to check the integrity of the message. The step-by-step process is performed to encode a message using the enhanced redundancy matrix encoding algorithm [19, 20].

The message is first converted into binary code and then divided into blocks of equal size. The redundancy matrix is created by adding additional rows and columns to the original message matrix. These additional rows and columns contain parity bits used. The matrix with the message, the redundancy matrix are multiplied together to produce the encoded message. Coded messages are sent or saved.

Using PCM, it becomes easier to identify the exact error location during data transmission. If the error is a single error, it can be corrected using the parity bit. If some errors are detected, the message may need to be sent. Overall, the error detection and correction code encoder using the improved or enhanced redundancy matrix code algorithm shows a suitable method for detecting and correcting errors in

messages. It is widely used in various applications such as communication systems, data storage systems, and digital media [21–23].

Consider an encoder was introduced that registers repeating bits using logic arrays and XOR inputs. Traditional parts are added to identify the defects. It is clearly indicated that from Fig. 1, horizontal bits and vertical bits are represented as $(H_{15}–H_0)$ and $(V_{15}–V_0)$, respectively, for generating the 32-bit input $(B_{31}–B_0)$.

The block diagram of IRMC encoder is shown in Fig. 1. The algorithm in the IRMC encoder starts with the same matrix. Choosing the logic conditions to calculate each equation and syndrome is not difficult when the equation matrix is chosen. For example, the AXL calculation uses symbols S_0 ($B_3B_2B_1B_0$) and S_2 ($B_{11}B_{10}B_9B_8$) to obtain the extra bits $[H_3H_2H_1H_0]$, which is shown in Fig. 2.

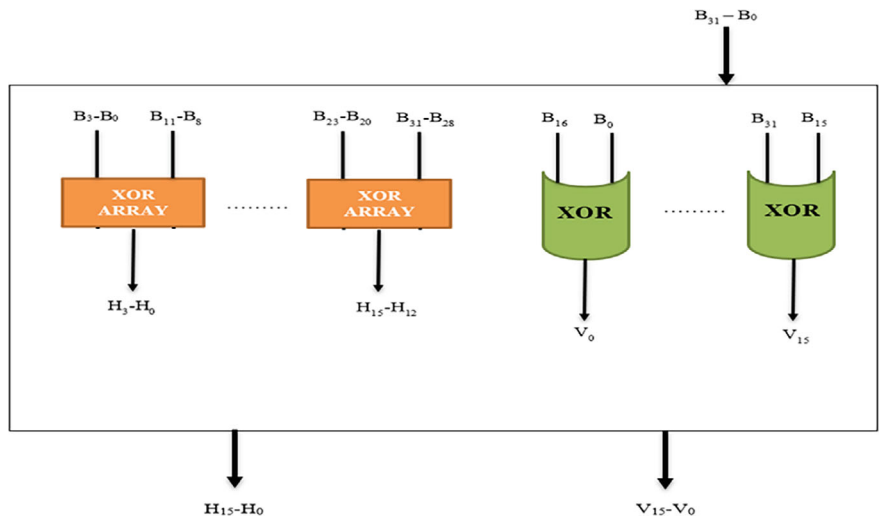


Fig. 1 Block diagram of IRMC encoder

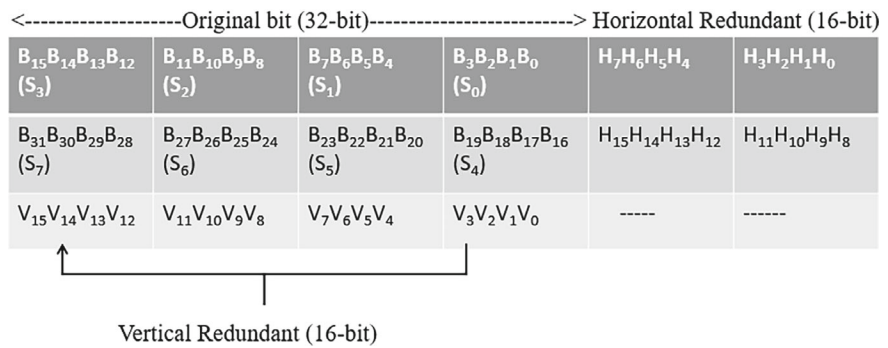


Fig. 2 Logical organization of original input and redundant bits

AXL, the horizontal and vertical redundant bits will be computed as shown:
Horizontal Bits generation using AXL calculation,

$$B_{11}B_{10}B_9B_8 \wedge B_3B_2B_1B_0 = H_3H_2H_1H_0 \quad (1)$$

$$B_{15}B_{14}B_{13}B_{12} \wedge B_7B_6B_5B_4 = H_7H_6H_5H_4 \quad (2)$$

Vertical Bits generation.

$$V_n = B_n \wedge B_n + 16 \quad (3)$$

where n = number of bits.

4 Decoder of IRMC Algorithm

The decoder is designed to detect and correct errors in messages using the improved redundancy matrix coding algorithm [24]. The encoder uses the parity check matrix generated during the encoding process to identify and correcting the errors in the received message. The encoding process begins with the received message, which is usually sent or stored as a binary string. The first step in the encoding process is to perform a similarity check on the received message using the similarity matrix created in the encoding process [25].

The equation matrix is a square matrix derived from the original message and the redundancy matrix [26]. This is used to check the involvement of the message by comparing the received message with encoded message. When the error is presented, the location and type of error can be predicted. If a bit error is detected, the decoder can correct the error by inverting the bit. When a greater number of errors are identified, the decoder may request that the message be retransmitted or use more advanced error correction techniques message [27, 28]. Detecting and correcting errors in messages provides an efficient and effective way to ensure accurate and reliable transmission. The schematic diagram of decoder is shown in Fig. 3.

About the terms used in Decoder,

B—Original Input (consider 32-bit)

H—Horizontal redundant bits (16-bit)

V—Vertical redundant bits (16-bit)

B^e—Erroneous Input (32-bit)

H^e—Horizontal redundant bits for erroneous input (16-bit)

V^e—Vertical redundant bits for erroneous input (16-bit)

S_p—Syndrome value (16-bit)

S_{pm}—Updated syndrome value (32-bit)

C—Final modified output with no error bits (32-bit)

The equations used in decoder are,

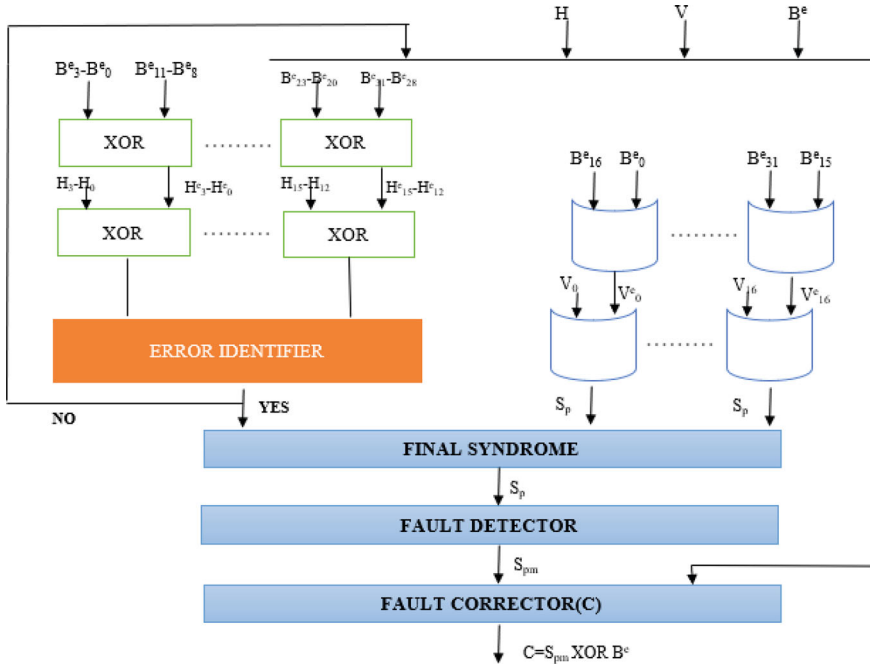


Fig. 3 Block diagram of IRMC decoder

i. Equation used for error identifier (YES/NO),

$$\text{if } H^e H^e = 0 \text{ no errors exists(NO)} \quad (4)$$

$$\text{if } H^e H^e \neq 0 \text{ error exists (YES)} \quad (5)$$

ii. Equation used to generate syndrome,

$$S_p = V^e V^e \quad (6)$$

iii. Equation used to result the error less output code,

$$C = S_{pm} \wedge B^e \quad (7)$$

To calculate the syndrome, the decoder performs mathematical operations on the received data with redundant bits, usually by multiplying by a predetermined matrix. This process generates a set of output values that can be compared against a predefined set of expected values to detect any deviations or errors. If the syndrome indicates that there is an error, the decoder can use several methods to correct the error. The syndrome is an important part of the error detection and correction process

in the code using the improved coding redundancy matrix algorithm. By calculating and analyzing syndromes, the decoder can identify and correct errors that may occur during transmission, ensuring that the data received is accurate and reliable.

5 Working of IRMC Algorithm for 32-Bit Input

Input Parameters & Flow of process to identify and rectify error bits, consider,

$$B = 1111\ 0101\ 1010\ 1111\ 1111\ 0110\ 1010\ 1100 \text{ (31st bit to 0th bit)}$$

Generating horizontal & vertical parameters for original input B

$$H = 0101\ 1010\ 0101\ 1010$$

$$V = 0000\ 0011\ 0000\ 0011$$

Consider, Erroneous input,

$$B^e = 1111\ 1101\ 1010\ 1101\ 1101\ 0100\ 0111\ 1100$$

Generating horizontal & vertical parameters for B^e

$$H^e = 0101\ 0000\ 1010\ 1000$$

$$V^e = 0010\ 1001\ 1101\ 0001$$

After obtaining H, H^e , V, and V^e

$$H \cdot H^e = 0000\ 1010\ 1111\ 0010 (\neq 0)$$

Since, error exists, we need to calculate Syndrome (S_p).

S_{pm} is simply generated by writing 16-bit S_p two times in a row.

Using S_{pm} , we rectify the error to obtain errorless output using $C = B^e \wedge S_{pm}$.

Hence, $C = 1111\ 0101\ 1010\ 1111\ 1111\ 0110\ 1010\ 1100$ (31st to 0th bit).

Figure 4 shows how the error bits are getting corrected using 2D forms, and also describes what the errors are located in the erroneous input to understand how the process is executed. Syndrome is S_p , calculated by XORing($\wedge$) V and V^e .

6 Results and Discussion

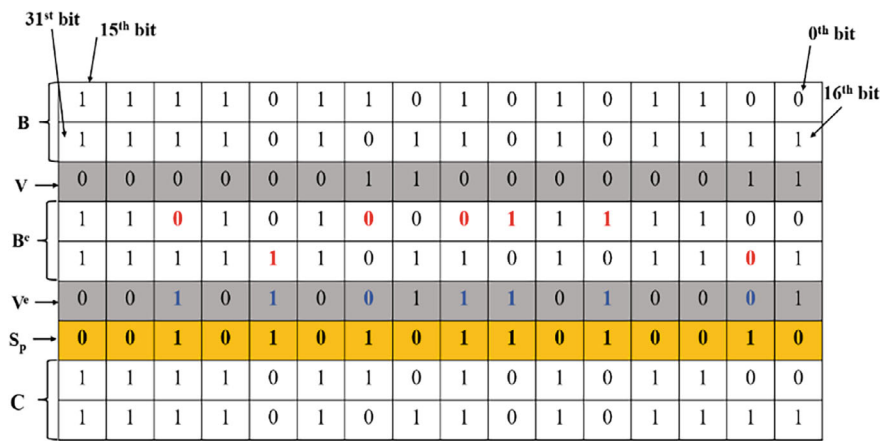


Fig. 4 Correction of error bits

Error Model Definition

The simulation and validation of the IRMC algorithm were conducted using synthetic fault injection models. However, the specific nature of these faults—whether random bit flips, burst errors, or radiation-induced SEUs/MCUs—has not been explicitly defined. For meaningful and reproducible evaluation, it is essential to clarify the error model used. Future work will incorporate standardized fault models such as SEU and MCU profiles based on radiation testing benchmarks. This will enable a more rigorous assessment of IRMC’s fault coverage and correction accuracy under realistic operating conditions.

6.1 Implementation of IRMC Algorithm Using Verilog HDL Language for 32-Bit Input

Simulation results, Design report & timing details of 32-bit encoded and decoded results are provided in Fig. 5, is shown. It also shows whether the erroneous 32-bit is getting equal to the original 32-bit after getting decoded.

6.2 Implementation of IRMC Algorithm for 64-Bit Input

Simulation results, Design report & timing details of 64-bit encoded and decoded results are provided in the coming figures (Fig. 6).

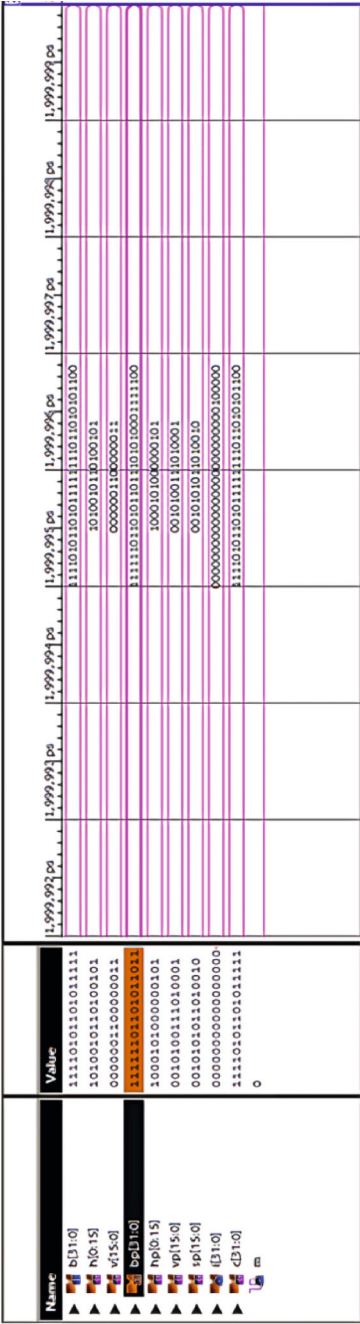


Fig. 5 Simulation result of 32-bit input

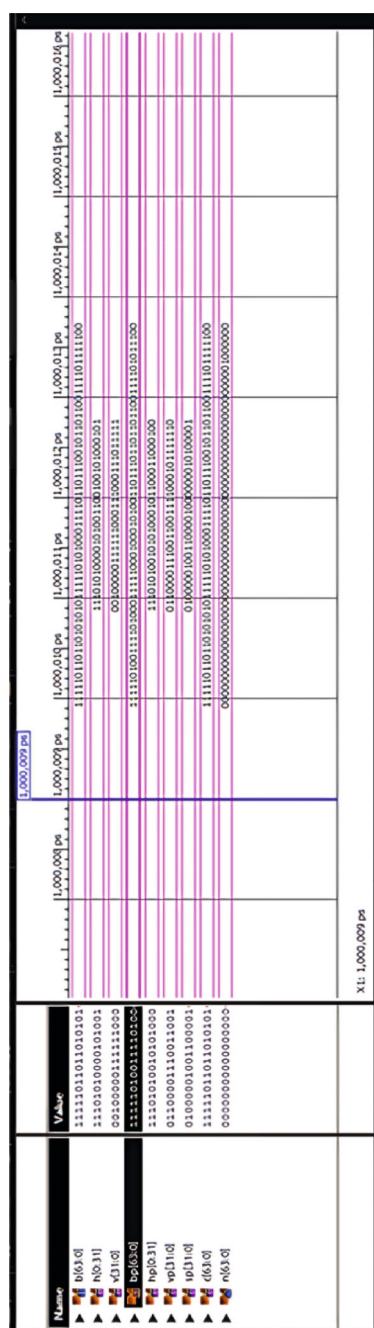
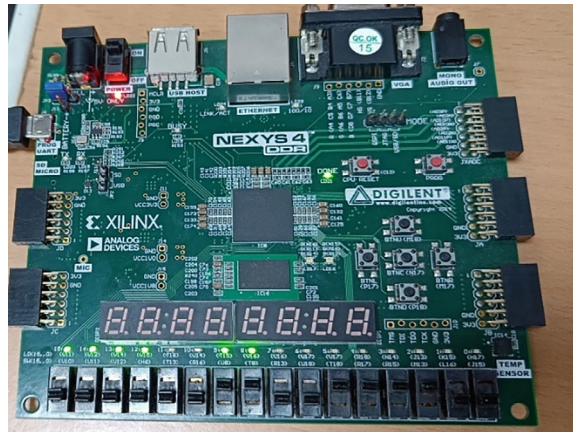


Fig. 6 Simulation result of 64-bit input

Fig. 7 FPGA implementation of IRMC



6.3 FPGA Implementation of IRMC Algorithm for 16-Bit Input

The FPGA kit used for implementation is Artix-7 XC7A100t board is shown in Fig. 7. A 16-bit input was taken as original input, and another 16-bit input was taken as the erroneous input. To take these inputs from FPGA board an array was considered which has four rows that can store 8-bits per each row. To normalize the clock, a clock divider is used to give input for a specific time lapse. In the similar manner, the 16-bit output also will be displayed in two specific time frames as 8-bits at a time. Based on application requirements PPA will be optimized. The error finding and correction will be made easy with the reconfigurable feature of FPGA.

Before proceeding to the discussion of the hardware resources utilization, it is important to note that the performance of the whole system is dependent upon the network architecture underlying it in general. Revolutionary types of designs like IRMC are usually built on top of targeted interconnection fabrics, e.g., Myo-Net, LoCoMo-Net, or other in-house topologies that directly influence the complexity of the routing, switching latency, and power. To capture parity in benchmarking and interpretation that would be valuable, future versions must clearly indicate the selected network and give the comparative perspective against the known structures in the literature.

6.4 Hardware Resource Utilization

The IRMC architecture was designed with resource efficiency in mind. The modular structure and streamlined control logic aim to minimize the use of LUTs and flip-flops, while avoiding excessive BRAM allocation. These design choices are expected to yield a compact footprint on FPGA platforms, particularly when scaled to larger

input sizes. Future work will include comprehensive synthesis reports to quantify resource utilization and enable direct comparison with existing implementations. These measures will allow both benchmarking of IRMC according to the current EDAC schemes and evaluation of its viability within a resource-poor setting. This will be expanded to 128-bit and 256-bit settings in the future.

6.5 Comparison of IRMC with State-of-the-Art Existing Algorithms

If we apply 32-bit data as input to the module, the delay value is 5.937 ns, and the power is 38.903 mW. The algorithms like DMC, BMC, OLS, etc., are similar to IRMC method. The simulation results of power and delay are compared with the above-mentioned methods.

Although the performance of IRMC in delay and power consumption is better than that of well-known and different techniques, i.e., DMC, BMC, and Reed-Muller codes, further benchmarking must be done through evaluation of the technique along with other dimensions. In particular, IRMC has an improved capability in correction that covers multiple bit errors compared to DMC and BMC, which are usually restricted to single-bit fault correction. Multi-bit correction is available in Reed-Muller codes, which have much greater decoding complexity and resource overhead.

In addition, IRMC has less algorithmic complexity and utilization of hardware resources; therefore, it is more suitable to run in real-time and embedded settings. These strengths make IRMC a well-balanced approach that integrates fault coverage, efficiency of computation, and feasibility of implementation features that are not covered by the traditional error correction schemes. Comparison Graphs for delay & power for 32-bit input are shown in Figs. 8 and 9, respectively.

Comparison Graphs for delay and power for 64-bit input are shown in Figs. 10 and 11, respectively.

7 Conclusion

The Improved Redundant Matrix Code (IRMC) algorithm presented in this work offers a novel approach to error detection and correction tailored for high-density VLSI systems. Unlike conventional schemes such as DMC, BMC, and Reed-Muller codes, IRMC introduces a dual-dimensional parity structure that enhances fault localization and correction precision while minimizing power, area, and timing overheads.

The novelty of IRMC lies in its ability to simultaneously achieve low-latency correction, high fault coverage, and hardware efficiency—a combination not

Fig. 8 Comparison graphical representation of delay analysis between IRMC (proposed) and other existing algorithms for 32-bit input

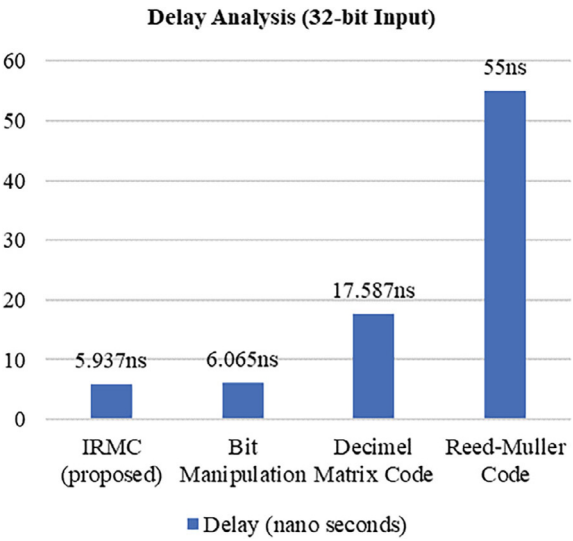
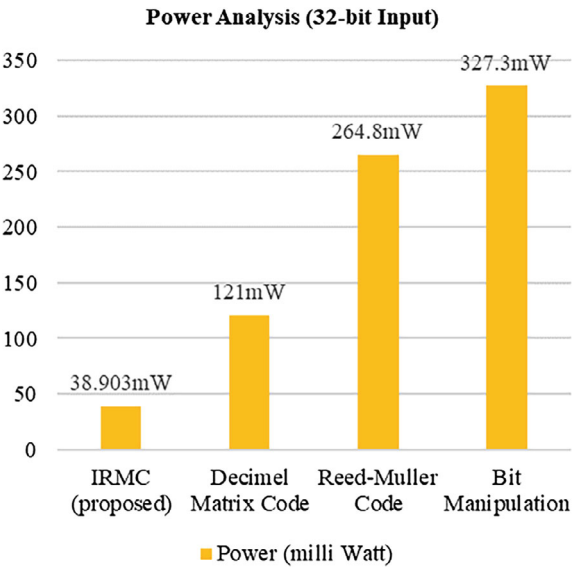


Fig. 9 Comparison of graphical representation of power analysis between IRMC (proposed) and other existing algorithms for 32-bit input



adequately addressed by existing methods. Through Verilog HDL implementation and FPGA-based validation, the proposed algorithm demonstrates superior performance in both 32-bit and 64-bit configurations, confirming its potential for deployment in next-generation embedded and communication systems.

Fig. 10 Comparison graphical representation of delay analysis between IRMC (proposed) and other existing algorithms for 64-bit input

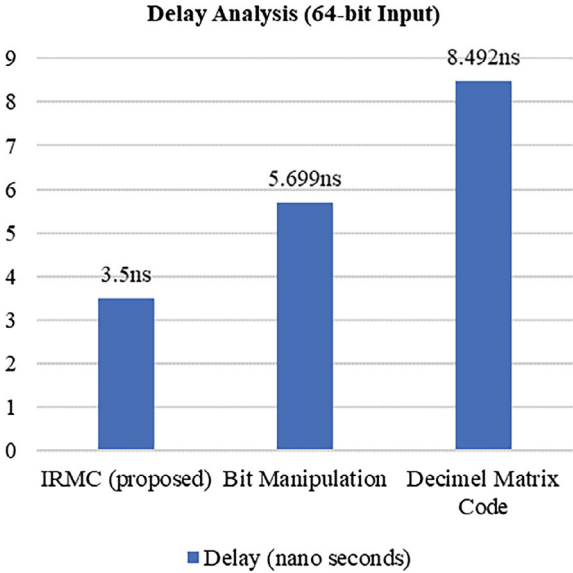
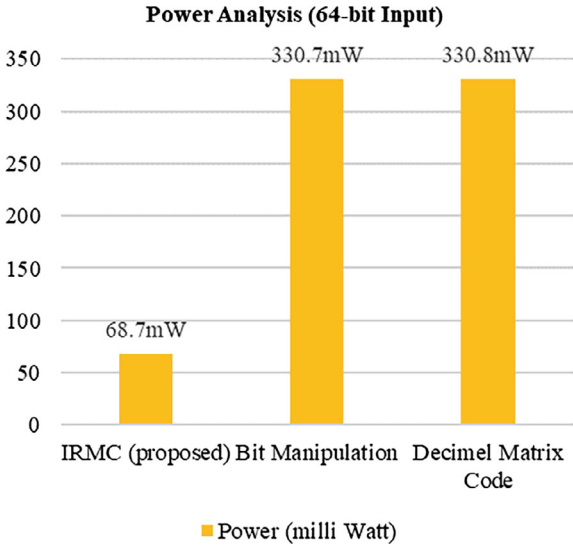


Fig. 11 Comparison of graphical representation of power analysis between IRMC (proposed) and other existing algorithms for 64-bit input



These results substantiate IRMC as a scalable and energy-efficient solution for safeguarding digital storage and transmission against soft errors, particularly in environments prone to radiation and electromagnetic interference.

7.1 Future Scope

As advanced technologies such as 5G networks, IoT devices, and space-grade electronics evolve rapidly, the demand for smart, low-power, high-performance fault correction techniques is more urgent than ever. Future research directions for IRMC include:

- Scalability to wider data buses: Extending IRMC to support 128-bit and 256-bit architectures is essential for compatibility with modern VLSI and embedded systems. This will involve optimizing parity matrix generation and syndrome decoding logic to maintain low-latency and power efficiency despite increased bit-widths. FPGA resource utilization, timing closure, and fault coverage trade-offs will be analyzed to ensure IRMC remains viable for real-time and data-intensive applications.
- Real-time deployment: Porting IRMC to real-time workloads such as autonomous systems, edge computing nodes, and mission-critical embedded platforms.
- Integration with emerging semiconductor technologies: Adapting IRMC for FinFET, SOI-based designs, and potentially quantum-resistant digital logic to ensure future-proof reliability.
- EDA toolchain compatibility: Automating synthesis and optimization of IRMC within modern EDA flows to facilitate deployment across diverse hardware platforms.
- Radiation resilience testing: Evaluating IRMC's performance in radiation-prone environments such as aerospace and satellite communication systems to validate its robustness under extreme conditions.

With the ever-growing technological landscape, IRMC's flexibility and configurability make it a promising asset for ensuring data integrity and system longevity in next-generation digital systems.

Acknowledgements Authors would like to acknowledge the Department of ECE, VNRVJIET, for providing the necessary computational tools.

Funding No external funding is involved in this developmental project.

Data Availability Authors declare no conflict of interest.

Declaration **Conflict of interest** The authors have not disclosed any competing interests.

References

1. Priya M, Vijay MM (2019) Error detection and correction for SRAM systems using improved redundant matrix code. In: International conference on recent advances in energy-efficient computing and communication (ICRAECC), Nagercoil, India, pp 1–8

2. Jenila R, Supraja C, Pappa CK, Dharani N (2021) VLSI implementation of error detection and correction codes for space engineering. In: 2021 third international conference on intelligent communication technologies and virtual Mobile networks (ICICV), Tirunelveli, India, pp 569–573
3. Liu B (2010) Error-detecting/correcting-code-based self-checked/corrected/timed circuits. In: NASA/ESA conference on adaptive hardware and systems, Anaheim, USA, pp 66–72
4. Naeimi H, DeHon A (2009) Fault secure encoder and decoder for Nano memory applications. *IEEE Trans Very Large-Scale Integr (VLSI) Syst* 17(4):473–486
5. Kaviya Priya K, Palaniappan M (2019) Implementation of error detection and correction codes using VLSI. *SSRG Int J Electron Commun Eng (SSRG-IJECE)* —Special Issue ICRTCRET
6. Schulz P, Tral A, Barreto AN, Fettweis G (2021) Efficient and reliable wireless communications via multi-connectivity using Riteless codes in single-and multi-user scenarios. *IEEE Trans Wirel Commun* 20(9):5714–5729
7. Hashemian FM, Shivaie M, Daneshthalab A (2019) A low-complexity and high-performance error correction code for non-volatile memories. *IEEE Trans Very Large Scale Integr (VLSI) Syst* 27(10):2225–2236
8. Chen Y, Zhang H, Song Y, Chen D (2019) Low-latency polar codes for HARQ feedback in 5G systems. *IEEE Trans Veh Technol* 68(6):5641–5645
9. Abinaya E, Somasundareswari D, Prasad SM (2015) Error detection and correction in 32-bit SRAM based FPGA memory using decimal matrix code. *Int J Sci Technol Manag* 04(1):2394–1529
10. Bhanumathi V, Sunandini M (2019) VLSI architecture for error detection and correction based on XOR against multiple cell upsets with reduced redundant bits. *ICTACT J Microelectron* 5(2):1680–2395
11. Yatskiv V, Kulyna S, Bykovyy P, Maksymyuk T, Sachenko A (2020) Method of reliable data storage based on redundant residue number system. In: IEEE 5th international symposium on smart and wireless systems within the conferences on intelligent data acquisition and advanced computing systems (IDAACS-SWS), Dortmund, Germany, pp 1–4
12. Le TP, Trong PH, Dang TQ (2020) High-throughput syndrome-based error correction codes for NAND flash memory. *IEEE Trans Very Large Scale Integr (VLSI) Syst* 28(9):2116–2120
13. Heidarzadeh H, Salami SE, Rashvand HF (2020) Optimized polar codes for beyond-5G terahertz communication systems. *IEEE Trans Commun* 68(2):1186–1195
14. Wang T, Liu H, Lin J, Wang C (2020) An efficient and low-complexity error correction scheme for non-volatile memory. *IEEE Trans Very Large Scale Integr (VLSI) Syst* 28(5):1139–1143
15. Zhou J, Liu Z, Li Y (2019) An efficient algorithm for low-density parity-check codes based on min-sum message-passing. *IEEE Trans Commun* 67(6):4224–4234
16. Xiao M, Liu M, Li H, Li Y (2020) Polar codes for channel coding in DNA storage systems. *IEEE Trans Nano Biosci* 19(2):179–186
17. Mathew NP, Mohan A (2016) Matrix code based error correction for LUT based cyclic redundancy check. *Procedia Technol* 25:590–597
18. Shanthi G, Rao KS, Kumar AS, Vali SS, Sravani KG, Hasan MM et al (2024) Multi-bit error detection and correction codes implementation using generator-parity check matrices technique. *Microsyst Technol* 1–12
19. Shanthi G, Vishwakanth Y, Teja LD, Kumar PA, Kumar AS (2023) Design of low-Power Turbo Coder for low-energy Mobile communications. In: 2023 14th international conference on computing communication and networking technologies (ICCCNT). IEEE, pp 1–7
20. Nagaleela S, Shanthi G, Manisha B, Bharath P, Praneeth E (2023) Design of DADDA multiplier using high performance and low power full adder. In: In 2023 14th international conference on computing communication and networking technologies (ICCCNT). IEEE, pp 1–5
21. Rajesh G, Yamuna B, Thirumala B, Kumar BKM, Gopi AV (2024) Space-grade VLSI implementation: error detection and correction codes in advanced aerospace systems. *Int J Mod Trends Sci Technol* 10(2):354–361
22. Noor Mohammed A, Satyanarayana J (2024) VLSI implementation of multi-bit error detection and correction codes. *J Emerg Sci Eng* 15(11):38–45

23. Manivannan T, Basheer Baba Y, Anish Kumar B, Narasimha Siva D (2024) Advanced VLSI technique for error detection and correction in space systems. *Int J Innov Sci Eng Appl* 5(2):112–119
24. Perez L, Kim J, Singh B (2024) Advanced ECC implementations in low-power embedded systems. *ACM Trans Embed Comput Syst* 23(3):457–472
25. Das S, Kumar V, Banerjee T (2024) Efficient parity-based error detection in high-speed communication networks. *Springer J Electron Test* 40(1):99–112
26. Sharma M, Gupta A, Verma R (2024) Optimized syndrome-based error detection for SRAM protection. *Elsevier Microelectron J* 128:211–223
27. Chatterjee P, Srinivasan K (2024) Multi-layer error correction for space-grade VLSI systems. *IEEE Trans Aerosp Electron Syst* 60(2):342–356
28. Shanthi G, Vali SS, Sai UB, Kumari KR, Leela SN, Rao KS (2025) Efficient error detection and correction code based on improved redundant matrix code (IRMC) algorithm. In: 2025 devices for integrated circuit (DevIC). IEEE, pp 802–807